Dedeepya Majety: EY GDS Technical & Architectural Interview Dossier

Target: EY Global Delivery Services - AI and DATA - Azure Data Engineer - Staff (Req ID: 4477576256)

Candidate: Dedeepya Majety
Current Role: Associate - Data Engineer @ Cognizant (3.9 YOE)
Target Level: Staff / Senior Data Engineer (Consulting - AI & DATA)
Location Preferences: Bengaluru / Hyderabad / Kolkata / Chennai
Notice Period Status: 🟢 Serving Official Notice Period · Available in 30 Days
Compensation Floor Anchor: Confirmed Offer @ 10.9 LPA · **Target Range:** 16.0 – 20.0+ LPA
Verified Credentials: Databricks Certified Data Engineer Associate, Microsoft Certified PL-300, Microsoft Certified PL-900

---

1. Executive Positioning & Recruiter/HM Opening Hook

1.1 The 60-Second Verbal Opening Hook (Memorize & Deliver Naturally)

> *"I am an Azure Data Engineer with nearly 4 years of experience at Cognizant, specializing in building and modernizing enterprise lakehouse platforms on Azure Databricks, PySpark, and Delta Lake.*
>
> *Most recently, I played a key role in modernizing 40+ legacy ETL workflows from Informatica PowerCenter to an Azure Databricks Medallion Lakehouse. This migration cut pipeline runtimes by 40%, lowered cloud compute spend by ~30%, and maintained 100% data reconciliation across millions of records. In parallel, because this EY GDS role specifically sits within the **AI and DATA Metadata Management** practice, my hands-on background managing **Informatica MDM match-merge trust scores, golden record survivorship, and Databricks Unity Catalog governance** makes me directly aligned with your team's immediate delivery needs.*
>
> *I am certified as a Databricks Data Engineer Associate and Microsoft Power BI Data Analyst. I am currently serving my official notice period with a confirmed offer in hand at ~11 LPA and can join EY within 30 days."*

---

2. EY GDS Practice Context & Alignment

EY GDS Practice Dimension	EY Expectation	Dedeepya Production Proof Point (Cognizant)
Domain Focus	Metadata Management, Data Governance, Data Quality	Informatica MDM match/merge logic, deduplication of 15% redundant entities, Databricks Unity Catalog 3-level namespace.
Core Modernization	Legacy DW/ETL to Modern Lakehouse (Databricks)	Migrated 40+ Informatica PowerCenter workflows to PySpark + Delta Lake, cutting batch runtimes by 40%.
Pipeline Reliability	Automated, resilient cloud orchestration	Apache Airflow DAGs with retries, SLA alerting, and ADLS Gen2 landing zone monitoring sustaining 99.8% uptime.
Analytics Enablement	Curated star-schema models for Power BI / Reporting	Gold Delta tables optimized with Z-Ordering and liquid clustering, powering 6+ executive Power BI dashboards with 25% query latency drop.

---

3. Top 10 Technical Viva & Architecture Defense Questions for EY GDS

Q1: EY GDS places heavy emphasis on Metadata Management. How do you distinguish between Technical, Operational, and Business Metadata in a Databricks Lakehouse?

Structured Viva Answer: - **Technical Metadata:** Describes the structural schema and physical storage. In Delta Lake, this is stored inside the `_delta_log/` JSON commit files and Unity Catalog metastore: schema definition, data types, column nullability, partition columns, file format, and compression codecs (Snappy Parquet). - **Operational Metadata:** Captures execution telemetry, pipeline health, and data movement: job run IDs, execution timestamps, duration, rows read/written/updated, commit version numbers, Spark task metrics, and Airflow DAG state history. - **Business Metadata:** Adds enterprise context and semantic understanding: column descriptions, business glossary terms, data classification (PII, Confidential, Financial), data ownership, and retention policies mapped via Unity Catalog tags.

Q2: How does Databricks Unity Catalog govern metadata, lineage, and access compared to legacy approaches?

Structured Viva Answer: - **Unified 3-Level Namespace:** Replaces detached metastores with `catalog.schema.table` (or view/volume). - **Automated Column-Level Lineage:** Unity Catalog captures runtime lineage from PySpark and SQL queries down to the column level without requiring third-party agents or code annotations. If a Silver column is computed from two Bronze columns, Unity Catalog traces the exact dependency graph. - **Centralized Fine-Grained Security:** Uses standard ANSI SQL to manage table, row, and column permissions: ```sql -- Column Masking CREATE MASK ssn_mask ON TABLE sales.customers(ssn) RETURN CASE WHEN is_account_group_member('compliance') THEN ssn ELSE 'XXX-XX-' || RIGHT(ssn, 4) END;

-- Row-Level Filtering CREATE FILTER region_filter ON TABLE sales.orders(region) RETURN is_account_group_member(region); ``` - **Cross-Workspace Sharing:** Delta Sharing allows secure data sharing with external partners and clients without duplicating data across storage accounts.

Q3: You worked with Informatica MDM. How do Match and Merge rules work, and how do you handle entity survivorship?

Structured Viva Answer: 1. **Tokenization & Search Keys:** Informatica MDM generates match tokens (keys) on attributes like Customer Name, Tax ID, Phone Number, and Address using fuzzy phonetic algorithms (e.g., soundex or meta-phone). 2. **Match Rules:** - **Deterministic (Exact Match):** E.g., Exact match on Government Tax ID / SSN or National ID. If matched, assigned 100% confidence. - **Probabilistic (Fuzzy Match):** E.g., Fuzzy match on Name (Jaro-Winkler distance > 0.88) combined with Address and Phone number. 3. **Trust Scores & Validation Rules:** Each source system is assigned a trust score for each attribute (e.g., CRM might have higher trust for Address, while Billing has higher trust for Bank Account). Over time, trust decays based on last update timestamp. 4. **Survivorship & Golden Record Generation:** When two records merge, the Master Record (Golden Record) takes the highest-trust value for each individual attribute, creating a single unified customer profile. 5. **Cognizant Production Impact:** We resolved conflicting records across enterprise acquisitions, eliminating ~15% duplicate master entities and preventing redundant customer billing.

Q4: When migrating 40+ legacy Informatica PowerCenter workflows to Databricks PySpark, how did you guarantee 100% data parity and zero regression?

Structured Viva Answer: - **Step 1: Transformation Decomposition:** Deconstructed legacy Informatica Mappings (Source Qualifier, Expression, Lookup, Aggregator, Router, Update Strategy) into modular PySpark transformation steps. - **Step 2: Automated Dual-Run Ingestion:** Ran both Informatica and PySpark pipelines in parallel against production mirror data for a 2-week validation cycle. - **Step 3: Parity Testing Framework:** Authored PySpark validation scripts comparing: - Exact row counts by partition and date. - Checksum hashing: computed SHA-256 hashes across concatenated non-temporal columns (`hash(concat_ws("||", *data_cols))`) and performed outer joins to isolate mismatched rows. - Precision and rounding verification for decimal/currency columns. - **Step 4: Cutover & Rollback Safety:** Configured Delta Lake Time Travel so that if any downstream consumer flagged a

regression, we could restore previous versions instantly with `RESTORE TABLE silver_table TO VERSION AS OF <version>`.

Q5: How do Delta Lake Schema Enforcement and Schema Evolution prevent data corruption while allowing business agility?

Structured Viva Answer: - **Schema Enforcement (Defense):** Delta Lake verifies on every write that incoming DataFrame schema matches the target table schema. If an incoming record contains an unexpected column or incompatible data type (e.g., String into Integer), Delta aborts the write with `AnalysisException`, preventing bad data from corrupting downstream models. - **Schema Evolution (Agility):** When a source intentionally adds new fields, we enable `.option("mergeSchema", "true")`. Delta automatically alters the target table metadata to append the new columns while backfilling existing rows with `NULL`. - **Auto Loader Handling:** In Auto Loader (`cloudFiles`), setting `.option("cloudFiles.schemaLocation", path)` dynamically discovers schema changes and automatically dumps corrupt or unparseable fields into `_rescued_data` without crashing the pipeline.

Q6: How do you design an end-to-end Medallion Architecture on Azure?

Structured Viva Answer: - **Bronze (Raw / Append-Only):** - Ingestion via Auto Loader: `spark.readStream.format("cloudFiles").option("cloudFiles.format", "json").load(adls_landing_path)`. - Retains raw payload, source file metadata, and ingestion timestamp. Never alters source values. - **Silver (Cleansed / Conformed / Enriched):** - Cleansed data: data types cast, null handling, deduplication using window functions (`row_number() OVER (PARTITION BY id ORDER BY updated_at DESC)`). - Delta ACID `MERGE INTO` (Upsert): updates matching existing records and inserts new records. - **Gold (Curated / Star Schema / Aggregates):** - Dimensional models (Fact and Dimension tables) optimized for BI queries. - Liquid clustering or Z-Ordering on primary join/filter keys (`OPTIMIZE gold_sales ZORDER BY (customer_id, transaction_date)`). - High-performance consumption via Databricks SQL Warehouse and Power BI.

Q7: What is the difference between Z-Ordering and Liquid Clustering in Delta Lake?

Structured Viva Answer: - **Z-Ordering:** - Traditional multidimensional clustering algorithm that maps multidimensional data into a 1D space using space-filling curves. - Requires manual execution of `OPTIMIZE table ZORDER BY (colA, colB)`. - Drawback: Performance degrades if data is heavily appended without frequent rewrites, and you cannot change Z-Order columns easily without rewriting the whole dataset. - **Liquid Clustering (Delta 3.0+ / Databricks):** - Replaces traditional partitioning and Z-Ordering with flexible, incremental clustering. - Syntax: `CREATE TABLE table_name CLUSTER BY (colA, colB);`. - Benefits: Incremental (only clusters newly added files during `OPTIMIZE`), no data skew caused by high-cardinality partitions, and allows altering clustering keys dynamically (`ALTER TABLE table_name CLUSTER BY (colC)`) without full table rewrites.

Q8: How did you implement Slowly Changing Dimensions (SCD Type 2) in Delta Lake?

Structured Viva Answer: - **Goal:** Maintain historical tracking of customer address or status changes with `start_date`, `end_date`, and `is_current` flag. - **Execution Strategy:** 1. Filter incoming source batch for records where attributes changed compared to the active Silver records (comparing MD5/SHA256 hash of dimension attributes). 2. Perform a 2-step single-transaction `MERGE INTO`: - **Match Condition:** Expire current row by updating `is_current = false` and `end_date = current_date()`. - **Insert Condition:** Insert the new record with `is_current = true`, `start_date = current_date()`, and `end_date = NULL`.

Q9: How do you handle pipeline orchestration and retries between Azure Data Factory (ADF) and Apache Airflow?

Structured Viva Answer: - **ADF vs Airflow Selection:** ADF is excellent for event-based triggers, secure file copies from on-prem/SFTP into ADLS Gen2, and zero-code integrations. Apache Airflow is superior for complex, code-driven orchestration, multi-hop dynamic dependencies, and custom Python validations. - **Production Airflow Setup at Cognizant:** - Modular DAGs using `DatabricksSubmitRunOperator` / `DatabricksRunNowOperator`. - Resiliency: `retries=3`, `retry_delay=timedelta(minutes=5)`, `retry_exponential_backoff=True`. - Task isolation: Each task represents an idempotent state. If a task fails on Silver, retrying does not duplicate data

because of Delta ACID `MERGE INTO`. - Telemetry: Integrated Slack/Email webhook alerts on SLA breaches or task failure.

Q10: How do you design Power BI reporting datasets over Delta Lake? (PL-300 Alignment)

Structured Viva Answer: - **Storage Modes:** - **Import Mode:** Fast in-memory DAX queries (VertiPaq engine), scheduled cache refreshes. Best for aggregated executive dashboards (< 1GB). - **DirectQuery Mode:** Queries passed directly to Databricks SQL Warehouse at runtime. Zero data duplication; real-time data freshness; performance depends on SQL warehouse compute. - **Direct Lake Mode (Microsoft Fabric / OneLake):** Directly reads Delta Parquet files into the Power BI engine without import translation or DirectQuery latency. - **Optimization Strategy:** - Pre-aggregate heavy metrics in the Gold Delta layer. - Establish a clean Star Schema (1 Fact table surrounded by 1:Many Dimensions). - Avoid bidirectional cross-filtering to keep DAX query plans optimal.

---

4. Live Coding Defense Drills (PySpark & SQL)

Drill 1: Deduplication with Window Functions in PySpark

```
from pyspark.sql.window import Window
import pyspark.sql.functions as F

# Deduplicate incoming customer records by keeping the latest update
window_spec = Window.partitionBy("customer_id").orderBy(F.col("update_timestamp").desc())

deduped_df = (
    raw_df
    .withColumn("row_num", F.row_number().over(window_spec))
    .filter(F.col("row_num") == 1)
    .drop("row_num")
)
```

Drill 2: Idempotent Delta Lake MERGE INTO (Upsert)

```
MERGE INTO silver.customer_master AS target
USING staged_updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED AND target.record_hash <> source.record_hash THEN
  UPDATE SET
    target.customer_name = source.customer_name,
    target.email = source.email,
    target.address = source.address,
    target.record_hash = source.record_hash,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN
  INSERT (customer_id, customer_name, email, address, record_hash, created_at, updated_at)
  VALUES (source.customer_id, source.customer_name, source.email, source.address, source.record_hash, curr
```

---

5. Reverse Inversion Questions to Ask EY GDS Interviewers

1. *"How is EY GDS standardizing metadata catalogs and data lineage across multi-cloud client engagements: is the primary standard Databricks Unity Catalog, Microsoft Purview, or custom metadata frameworks?"*
2. *"Given that this team sits in AI and DATA Consulting, how frequently do your delivery teams lead legacy-to-modern lakehouse migrations versus greenfield platform builds?"*
3. *"What does the CI/CD and deployment lifecycle look like for Databricks notebooks and Airflow DAGs across your client delivery environments?"*

4. *"What is the single biggest architectural challenge this metadata management practice is currently navigating across its enterprise client portfolio?"*

---

6. Compensation & Notice Period Strategy for EY GDS

- **Current Status:** Officially serving 30-day notice period at Cognizant.
- **Anchor Offer:** 10.9 LPA (Infosys offer confirmed).
- **Target EY Compensation:** 16.0 – 20.0 LPA.
- **Script:** *"I am currently serving my official notice period at Cognizant and can join EY within 30 days. I have already cleared recruitment pipelines with a confirmed offer in hand at ~11 LPA. Based on my 3.9 years of direct enterprise delivery in Databricks PySpark and metadata management, along with my Databricks and Power BI certifications, I am targeting between 16 and 18 LPA for this role."*